

Comisión 102340

# Scrum 4.0

**CODERHOUSE**

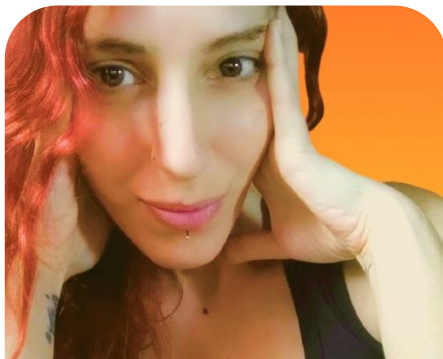
**¡Esta clase<sup>o</sup>  
va a ser  
grabada!**

# Agenda

1. Mindset ágil: Más allá de los procesos
2. Scrum vs. Waterfall: Eligiendo el enfoque adecuado
3. Product Discovery: Identificación del problema y propuesta de valor
4. Organización del Proyecto y Stakeholders
5. Pre-entrega: Brief de Proyecto y Mapa de Stakeholders
6. Resumen

# Presentación del equipo :

COMISIÓN N°102340



**Profesora:**  
Yanina Andregnette



**Profesora:**  
Melisa Braslavsky

# Proyecto final

Consignas

# Consigna

El entregable final consiste en una Presentación Ejecutiva o Documento Maestro que consolide la gestión completa del producto.

Deberás presentar un caso donde se visualice la trazabilidad total: desde el dolor del usuario hasta la métrica que indica que el problema fue resuelto.

## Estructura requerida del proyecto:

1. Visión y Estratégica: Resumen de la propuesta de valor y el MVP consolidado (actualizado según el feedback de los módulos 4 y 6).
2. Product Discovery y Validación: Consolidación de los experimentos realizados y el aprendizaje obtenido (Checkpoint M6).
3. Hacia la Ejecución (Scrum en Acción):
  - Product Backlog Refinado: Top 10 historias de usuario con criterios de aceptación y estimación.
  - Definition of Done (DoD): Tu "barra de calidad" clara para este proyecto específico.
4. Ecosistema de Herramientas y Flujo: Capturas o enlaces a tu tablero de Jira/Trello y Miro, explicando el workflow y las automatizaciones implementadas.
5. Cierre y Operaciones: Plan de lanzamiento y cómo medirás el éxito post-entrega (Métricas de flujo y de producto).

# Formato de Entrega

La entrega deberá realizarse mediante **un único enlace o archivo**.

Podrás utilizar:

- Google Docs
- Google Slides



Importante:

- Solo se puede entregar un único enlace o archivo.
- Todo el contenido del proyecto debe estar centralizado en ese documento.

## Criterios de evaluación

- La evaluación del proyecto final será realizada por **Ticher** y revisada por el equipo docente.
- El último día de la cursada se visualizarán las rúbricas de evaluación en plataforma.

# 01

Mindset ágil: Más allá de los procesos

# ¿Por qué nació la Agilidad?

La agilidad nació como respuesta a que las formas “clásicas” de gestionar proyectos no alcanzaban para entornos cambiantes.

**El contexto cambia:** en los proyectos reales (software y también fuera de TI) aparecen nuevos requisitos, prioridades y aprendizajes en el camino.

**Los planes rígidos fallan:** metodologías muy basadas en “cerrar todo de entrada” y seguir un plan detallado hacen difícil adaptarse cuando el escenario cambia.

**Se necesitaba aprender rápido y ajustar:** la agilidad propone entregas frecuentes, feedback continuo y colaboración para tomar mejores decisiones mientras el producto evoluciona.

# Los 4 valores

Agilidad, en términos simples, es una forma de trabajar en proyectos donde el objetivo no es “seguir un plan perfecto”, sino adaptarse rápido a lo que va pasando.

Se basa en valores compartidos (los del Manifiesto Ágil) que priorizan:



Personas y comunicación  
por encima de solo  
herramientas/procesos

Resultados reales (algo  
funcionando) antes que  
documentos extensos

Trabajo conjunto con el  
cliente/usuario durante el  
proceso

Ajustar cuando cambia el  
contexto en vez de  
apegarse rígidamente al  
plan

# ¿Qué es el Manifiesto Ágil?

El Manifiesto Ágil es un documento que resume valores y principios para trabajar con proyectos de una manera más flexible.

Su idea central es que, en un entorno donde cambian los requisitos, lo más importante no es seguir procesos rígidos ni basarse únicamente en documentación, sino priorizar la comunicación entre las personas, la entrega de resultados que funcionen, la colaboración continua con el cliente y la capacidad de responder al cambio.

# ¿Qué son los principios ágiles?

Los Principios Ágiles son 12 pautas que traducen los valores del Manifiesto a la práctica.

No son reglas fijas: te orientan a tomar decisiones en el día a día para mejorar el trabajo en proyectos.

1. Priorizar al cliente con entrega temprana
2. Dar bienvenida al cambio
3. Entregar con frecuencia
4. Colaboración constante con el cliente
5. Personas motivadas y ambiente de apoyo
6. Conversaciones cara a cara (comunicación efectiva)
7. Medir el progreso con funcionamiento real
8. Mantener un ritmo sostenible
9. Atención a la excelencia técnica y buen diseño
10. Simplicidad (hacer lo necesario)
11. Equipos autoorganizados
12. Mejorar continuamente y ajustar

## 12 principios del Manifiesto Ágil



Satisfacción del cliente



Bienvenidos los cambios



Hacemos entregas frecuentes



Colaboración entre áreas



Colaboradores motivados



Comunicación cara a cara



Medición de avances



Promover el desarrollo sostenido



Atención a la excelencia y el buen diseño



La simplicidad es esencial



Equipo auto-organizado

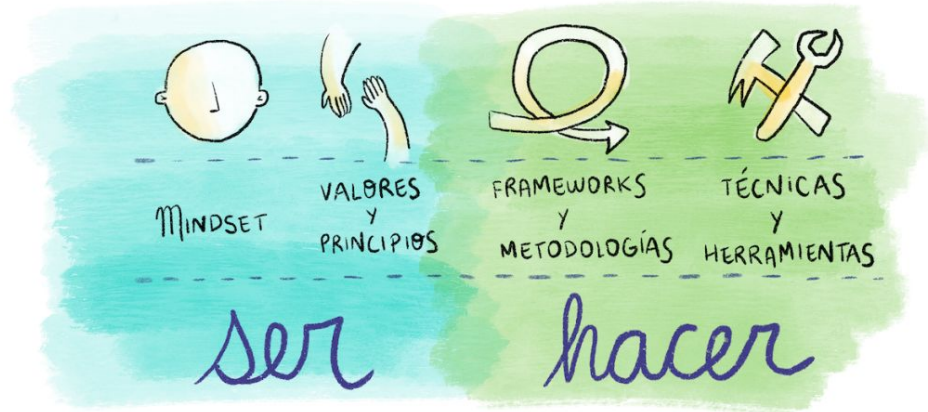


Reflexión sobre la mejora continua

# Mindset Ágil vs. "Fake Agile"

El Mindset Ágil es un cambio real: se centra en adaptarse, aprender y entregar valor, con colaboración y mejora continua.

En cambio, el "Fake Agile" es cuando se "usa" agilidad solo en la superficie: se hacen ceremonias y se cambia el nombre, pero se mantiene la mentalidad de control rígido, poca adaptación y decisiones "de escritorio", sin feedback real ni entrega frecuente.



## Ser ágil ≠ Hacer agile

# 02

Scrum vs. Waterfall: Eligiendo  
el enfoque adecuado

# Scrum vs. Cascada

Scrum y Waterfall son dos maneras distintas de gestionar proyectos. La diferencia principal está en cómo manejan el cambio y la incertidumbre.

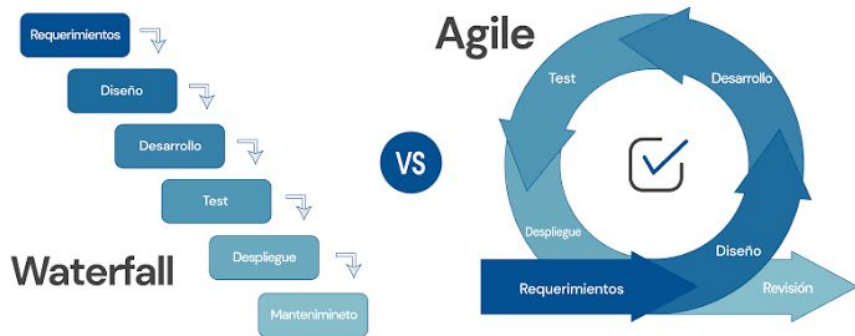
- **Waterfall** (enfoque predictivo) funciona como un plan en etapas: primero se define mucho del alcance y, a partir de eso, el proyecto avanza siguiendo una secuencia relativamente fija. Este enfoque suele servir cuando los requisitos están claros y estables, porque permite planificar con precisión y controlar el avance según el cronograma. El problema aparece cuando surgen cambios: si el contexto se mueve o el equipo aprende algo nuevo, adaptar el plan puede volverse caro y lento.
- **Scrum**, en cambio, asume que en muchos proyectos hay aprendizaje mientras se avanza. Por eso trabaja con ciclos cortos (sprints) en los que el equipo construye y entrega resultados en forma incremental. El plan no es “inmutable”: se va ajustando con base en feedback real del cliente/usuario y en lo que el equipo va entendiendo del trabajo. Scrum tiende a encajar mejor cuando hay incertidumbre, porque permite reducir el riesgo entregando antes, validando supuestos y corrigiendo rumbo a tiempo.

# Scrum vs. Cascada

Para elegir el enfoque adecuado, podés pensar en una regla simple:

- Si el objetivo y los requisitos son bastante conocidos, Waterfall puede funcionar.
- Si el camino no está del todo claro y es probable que cambien prioridades o se descubra información nueva, Scrum suele ser la opción más adecuada.

En ambos casos, la decisión busca que el proyecto sea controlable, pero con una diferencia: Waterfall apuesta a planificar al inicio; Scrum apuesta a aprender mediante entregas frecuentes.



# Actividad colaborativa

Scrum vs. Waterfall: Eligiendo el enfoque adecuado

Respondemos las preguntas de evaluación por el chat de Zoom o abriendo micrófono.



# Evaluación

## Pregunta 1

Estás gestionando un proyecto donde el cliente insiste en que todos los requisitos y diseños deben estar firmados y aprobados antes de que se escriba una sola línea de código. ¿A qué enfoque se alinea más esta restricción?

- A. Scrum, porque requiere que todo el backlog esté definido y aprobado antes de empezar el primer Sprint.
- B. Waterfall, porque se basa en fases secuenciales donde el diseño y los requisitos deben estar cerrados antes de la construcción.
- C. Ambos modelos requieren que el 100% de las funcionalidades estén detalladas contractualmente antes de ejecutar nada.

# Evaluación

## Pregunta 2

En un entorno de alta incertidumbre tecnológica (por ejemplo, integrando una IA nueva cuya capacidad real se desconoce), ¿cuál es la principal ventaja de utilizar Scrum sobre Waterfall?

- A. Scrum, porque optimiza para el aprendizaje y la gestión del riesgo mediante feedback temprano.
- B. Waterfall, porque permite predecir exactamente cuándo se terminará el proyecto desde el día uno.
- C. Scrum, porque al no tener documentación, el equipo puede programar mucho más rápido sin interrupciones.

# Evaluación

## Pregunta 3

¿Cuál es la diferencia fundamental en la 'entrega de valor' entre ambos modelos?

- A. En Waterfall el valor suele entregarse al final; en Scrum se entrega incrementalmente en cada Sprint.
- B. En Scrum el valor se entrega solo cuando el Product Owner decide cerrar el proyecto.
- C. En Waterfall el valor se mide por horas hombre, y en Scrum por el número de reuniones realizadas.

# Evaluación

## Pregunta 4

¿En qué escenarios podría Waterfall ser más adecuado que Scrum? (Selecciona todas las que apliquen)

- A. Un proyecto de infraestructura física con fechas legales de cumplimiento inamovibles.
- B. Un proyecto de cumplimiento normativo (Compliance) donde los requisitos legales son fijos y conocidos.
- C. El desarrollo de una aplicación móvil para una red social nueva con mercado incierto.

# Evaluación

## Pregunta 5

Un stakeholder afirma que no quiere usar Scrum porque 'en Scrum no se documenta nada'. ¿Cuál sería la respuesta correcta desde un nivel intermedio de gestión?

- A. Scrum promueve la documentación suficiente y con propósito, no la ausencia total de ella.
- B. Scrum prohíbe escribir cualquier documento que no sea el código fuente de la aplicación.
- C. En Scrum solo se documenta si el Scrum Master tiene tiempo libre entre reuniones.

# 03

**Product Discovery:  
Identificación del  
problema y propuesta de  
valor**

# El corazón del Product Discovery: El Problema ante todo

Primero se busca entender bien el problema antes de pensar en “la solución”.

La idea es que muchas veces el equipo cree que tiene clara la necesidad del usuario, pero en realidad lo que está faltando es responder: ¿Qué dolor real estamos resolviendo y para quién?

Si el problema está mal definido, aunque construyas “algo” con buenas intenciones, es probable que no genere valor.

# El Problema ante todo

En la práctica, el Product Discovery empieza con preguntas como:

- ¿Qué situación está atravesando el usuario?
- ¿Qué hace hoy para resolverlo (o qué le cuesta)?
- ¿Por qué es un problema importante ahora?
- ¿Cómo medimos que se mejoró (valor/resultado)?

Así, “el corazón” es concentrar la energía en validar el problema y sus causas. Recién después se formula una propuesta de valor que apunte a resolverlo.

# Identificación de Actores: ¿Para quién resolvemos?

Identificar actores en Product Discovery es responder “¿para quién resolvemos este problema?”. No alcanza con decir “usuarios” de forma general: hay que entender quiénes participan en el problema y cómo se relacionan con la situación.

En la práctica, los actores incluyen normalmente:

- El usuario principal (quien vive el dolor o necesita el cambio).
- Quien toma decisiones (puede no ser la misma persona: por ejemplo, compra/autoriza).
- Quien usa el producto en el día a día (a veces es otro perfil).
- Stakeholders que influyen o se ven afectados (soporte, operaciones, etc.).

## ¿Por qué esto es clave?

Porque la propuesta de valor tiene que ser relevante para el actor correcto.

Si apuntas al “usuario”, pero en realidad quien decide tiene otras prioridades, terminas resolviendo el problema equivocado o comunicando mal el valor.

# Jobs To Be Done: Descubriendo qué "contrata" el cliente

Es la situación o tarea concreta que el cliente quiere resolver para obtener un resultado. No habla de “necesidades” en general, sino del trabajo que la persona intenta hacer en su día a día. Sirve para entender el problema desde el lado del usuario: no como “necesidad” abstracta, sino como la tarea o situación que la persona intenta resolver.

La clave es que, muchas veces, el cliente “contrata” una solución para lograr un resultado específico. Ese “resultado” es el job.

Por eso, cuando investigamos, buscamos responder algo como: ¿qué estaba pasando antes? ¿qué intentaba lograr? ¿por qué ahora? ¿qué tendría que cambiar para que sea un éxito?

# Jobs To Be Done

Un buen JTBD suele describirse con esta estructura:

**Cuando** ocurre X situación...

**El/la usuario** quiere lograr Y resultado...

**Porque** necesita/valora Z (la razón o condición).

## Ejemplo:

“Cuando una persona está por salir y quiere evitar perder tiempo buscando cosas, quiere tener un listado claro de qué llevar porque quiere reducir el estrés y llegar a horario.”

Esto conecta directo con la propuesta de valor: si tu valor resuelve el job correcto, el producto tiene más chances de generar adopción y resultados.

# Break,

andá por un cafecito  
o mate, ¡y volvemos!



# Pains & Gains

Los **Jobs To Be Done** se conectan con **Pains & Gains** porque ambos expresan valor y dolor desde la perspectiva del usuario.

El *Job* describe **qué resultado “contrata” el cliente**. En ese marco, los *Pains* son los **obstáculos que le impiden completar el job** (qué sufre o qué le cuesta).

Los *Gains* son los **beneficios esperados al lograrlo** (qué espera mejorar o evitar).

Así, JTBD te da el “para qué”, y Pains & Gains te ayuda a formular la propuesta de valor con el lenguaje del usuario.



# Diseño de la Propuesta de Valor Digital

Una propuesta de valor digital no es un eslogan: es la explicación de por qué el cliente te elegiría en lugar de las alternativas, incluso la de “no hacer nada”.

En productos digitales suelen aparecer “cosas que la app hace” (features).

Pero para el cliente eso no alcanza, porque lo que decide es lo que le provoca usarlo.

- **Feature:** una capacidad del producto (ej. “sincroniza en tiempo real”).
- **Benefit:** lo que esa capacidad te resuelve o te mejora (ej. “evita confusiones entre personas”).
- **Outcome/resultado:** el efecto medible o el cambio final en el trabajo (ej. “menos errores y menos retrabajo”).

En vez de mostrar “qué hace la app”, explicá qué cambia para el usuario.

Tu propuesta de valor tiene que responder: “¿por qué vale la pena reemplazar lo que ya hacen?”

# Validación Ágil: Del supuesto a la evidencia

En Scrum, el objetivo no es “armar cosas” y después ver si sirven.  
Usamos Product Discovery porque buscamos convertir ideas en evidencia.

- Un supuesto es “creemos que pasa algo”.
- La evidencia es “esto está pasando / se observa en datos o en entrevistas”.

## ¿Qué evidencia buscamos?

Señales cualitativas (comprensión del porqué)

- Entrevistas con usuarios
- Tickets de soporte / quejas recurrentes
- Observar cómo trabajan hoy

Señales cuantitativas (comprobación con números)

- Tasa de drop-off (gente que abandona el proceso)
- Tiempo promedio para completar una tarea
- Tasa de errores en herramientas actuales (ej. CRM)

# Errores frecuentes

## Lo que nos hace perder tiempo y valor

### Hacer “cosas lindas” en vez de arreglar un dolor

“Sería bueno tenerlo” (no es urgente).

“Lo necesitamos porque está roto / molesta” (eso sí es prioridad).

### Ser demasiado general

“Mejora el trabajo”  
¿Cómo? ¿Cuánto mejora?  
Si no se puede medir, no se puede validar.

### No tomar en cuenta un “No” desde el principio.

Si alguien clave dice “no por seguridad / compliance / factibilidad” y vos no lo consideraste, el proyecto se frena aunque al usuario le encante.

# Buenas prácticas

## Cómo hacerlo bien

**Hablar como habla el cliente** (en vez de lenguaje técnico).

En vez de “optimizar latencia”, pensalo como “evito esperas”.

**Compararte con lo que el cliente hace hoy** (planillas, caos manual, etc.)

La propuesta debe explicar por qué tu opción es mejor que eso.

**Ajustar la propuesta** cuando aparezca evidencia (entrevistas / datos)

# 04

## Organización del Proyecto y Stakeholders

# El Ecosistema Humano: Mapeo de Stakeholders

Antes de armar el proyecto, tenés que saber quiénes están alrededor de la solución, porque no todos influyen igual.

## ¿Qué es un stakeholder?

Cualquier persona o área que:

- Usa el producto.
- Lo aprueba / decide.
- Lo gestiona.
- O afecta el resultado (aunque no lo use directamente).

# Mapeo de Stakeholders

## ¿Qué hay que mapear?

- Quién es (roles o áreas)
- Qué rol cumple:
  - Usa (usuario)
  - Decide (compra / aprobación)
  - Bloquea o permite (seguridad, IT, compliance)
  - Impacta (soporte, operación)
- Qué necesita / qué le importa (su prioridad)
- Qué tan importante es para el proyecto (influye: sí/no, y en qué punto)

# ¿Por qué importa mapearlos?

Porque en Agile (y sobre todo en Scrum) el trabajo se valida y se entrega con personas reales, y esas personas influyen en decisiones, prioridades y restricciones.

## **Reduce bloqueos:**

Si sabes quién puede frenar (seguridad, IT, compliance), anticipas requisitos y evitas “me encanta, pero no se puede”.

## **Alinea el Backlog:**

Te ayuda a ordenar qué pide el usuario, qué valida quien decide y qué es “condición de entrada”.

## **Mejora la validación:**

Los Sprints no validan solo “la app”, validan con quienes tienen información, permisos o criterio para juzgar el valor.  
Trabajo conjunto con el cliente/usuario durante el proceso

## **Evita suposiciones:**

Sin mapa, el equipo suele asumir que “el usuario” es también quien decide... y ahí aparecen fallas tarde.

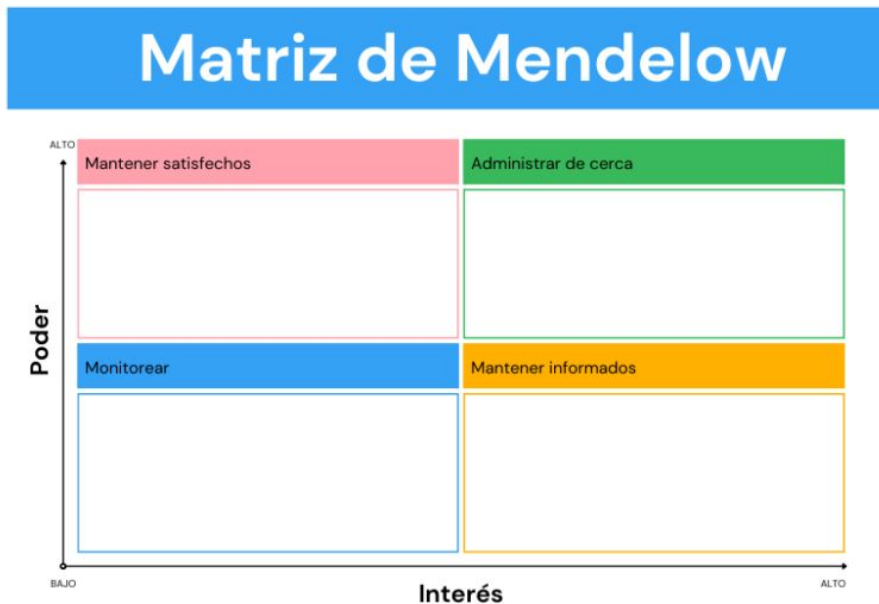
# La Matriz de Poder e Interés (Matriz de Mendelow)

**Qué es:** una forma de clasificar stakeholders según:

- Poder: Pueden aprobar o frenar.
- Interés: cuánto les importa el proyecto (seguimiento/participación).

**Cómo se usa:**

- Alto poder + alto interés: Gestionar de cerca (escucharlos seguido, involucrarlos en decisiones)
- Alto poder + bajo interés: Mantener satisfechos (informar lo justo)
- Bajo poder + alto interés: Mantener informados (consultarlos para entender necesidades)
- Bajo poder + bajo interés: Monitorear (información mínima)



# Gestión de Expectativas: El Arte de la Transparencia

En Agile no prometemos “magia”; prometemos transparencia.  
Así, stakeholders y equipo saben qué pasa, qué no está listo todavía y qué riesgo hay.

## **Product Backlog = herramienta de comunicación**

En vez de prometer fechas para “una lista infinita”, el Product Owner usa el Backlog priorizado para mostrar:

- Qué se entregará primero (prioridad alta)
- Qué queda para después (prioridad baja)
- Qué no entra por ahora (fuera del alcance actual)

## **Sprint Review vs Demo**

Error: Usar la Sprint Review como si fuera una demo para “mostrar y listo”.

Realidad: La Sprint Review es sesión de trabajo: los stakeholders miran lo entregado, dan feedback, si algo no les cierra, se detecta y se ajusta.

Esto es gestionar expectativas en tiempo real.

# Setup del Workspace Colaborativo: Construyendo una Single Source of Truth

Para que Agile funcione, la información tiene que ser transparente y fácil de encontrar.

Si cada quien trabaja con documentos distintos, versiones desactualizadas o chats dispersos, el equipo se traba en entender lo mismo, vuelve a preguntar y termina retrabajando.

Por eso se arma una *Single Source of Truth*: un lugar central donde vive lo importante del proyecto.

La idea no es “tener todo”, sino que todos consulten el mismo punto de referencia y con eso se alineen, decidan y ejecuten sin perder contexto.

Si lo construí en herramientas como Notion/Jira/Trello/Confluence/Google Workspace/Miro, lo realmente importante es el uso: criterios claros de organización, evitar duplicaciones, centralizar decisiones y que el equipo tenga acceso transversal.

*“No es la herramienta: es que todos trabajen con la misma verdad.”*

# 05

**Pre-entrega: Brief de Proyecto y Mapa  
de Stakeholders**

# Pre-entrega: Brief de Proyecto y Mapa de Stakeholders

## Checkpoint: Definición de Proyecto y Stakeholders

¿Qué debes construir?

Debes entregar un Documento de Definición Inicial (puedes usar la página que creaste en Notion) que contenga tres secciones críticas:

### 1. Declaración del Problema y Propuesta de Valor

No describas funciones (qué hace el producto), describe el impacto (qué problema resuelve).

- Segmento: ¿Para quién es esta solución?
- El "Dolor": ¿Cuál es el problema crítico que enfrentan hoy?
- Impacto del No-Hacer: ¿Qué pasaría si este problema no se resuelve en 90 días? (Costos, pérdida de usuarios, ineficiencia).
- Propuesta de Valor: Una frase que explique cómo tu solución digital alivia ese dolor de forma única.  
*Ejemplo: "Para equipos de operaciones que pierden 5 horas semanales en reportes manuales, nuestra herramienta automatiza la extracción de datos de Jira reduciendo el error humano a cero".*

# Pre-entrega: Brief de Proyecto y Mapa de Stakeholders

## 2. Mapa de Stakeholders (Matriz Poder/Interés)

Identifica al menos 5 interesados clave y ubícalos en una matriz de Poder vs. Interés. Debes incluir perfiles de:

- Negocio/Sponsor: Quién paga o aprueba.
- Operaciones/IT: Quiénes mantienen o integran la solución.
- Usuarios Finales: Quiénes usarán la herramienta a diario.
- Externos (opcional): Reguladores, clientes externos o partners.

## 3. Estrategia de Gestión de Expectativas

Para tus dos stakeholders más críticos (los de Alto Poder / Alto Interés), define:

- ¿Qué esperan ellos del proyecto? (Ej: "Reducción de costos", "Facilidad de uso").
- Frecuencia de comunicación: ¿Les basta con el Sprint Review o necesitan un reporte mensual de KPIs?

## 4. Riesgos Iniciales del Proyecto

Identificá al menos 3 riesgos potenciales:

- técnicos,
- operativos,
- organizacionales,
- o de adopción.

Y propone una posible estrategia de mitigación.

# Pre-entrega: Brief de Proyecto y Mapa de Stakeholders

## Pasos sugeridos

1. Entrevista o Empatía: Si tu proyecto es para tu empresa actual, habla con un potencial usuario por 15 minutos. Escucha sus quejas, no le vendas tu idea.
2. Lluvia de ideas de interesados: Anota a todos los que podrían decir "no" al proyecto o que podrían verse afectados por él.
3. Clasificación: Usa la matriz que vimos en la Unidad 4 para decidir quién necesita más atención.
4. Redacción: Escribe tu declaración del problema siguiendo la fórmula: *"Actualmente [Público] tiene el problema de [Dolor], lo que causa [Impacto Negativo]. Nuestra propuesta es [Solución] para lograr [Resultado medible]"*.

## Errores comunes a evitar

- Confundir "Features" con "Valor": Si tu propuesta de valor dice "haremos una app con login social", estás describiendo una característica. Cámbialo por "permitiremos que el usuario acceda en 2 segundos sin recordar contraseñas".
- Ignorar al stakeholder con poder de veto: Muchos proyectos fracasan porque el equipo ignoró a "Ciberseguridad" hasta la semana del lanzamiento. Identifícalos ahora.

## Ejemplo

# Resumen

- Mindset ágil: Más allá de los procesos: Agile no es solo “seguir ceremonias”, sino sostener una forma de trabajar centrada en alineación, adaptación y valor real, evitando que el proceso se vuelva burocracia.
- Scrum vs. Waterfall: eligiendo el enfoque adecuado: Scrum es ideal cuando hay aprendizaje e iteración: el plan se revisa con el tiempo. Waterfall sirve mejor cuando el alcance está muy definido y cambia poco.
- Product Discovery: identificación del problema y propuesta de valor: Antes de construir, se busca entender el problema: para quién es, cuál es el dolor, qué pasa si no se resuelve y cuál es la propuesta de valor (valor/impacto, no features).
- Organización del proyecto y stakeholders: Se define quiénes participan y qué rol tienen: negocio/sponsor, operaciones/IT, usuarios finales (y opcionales externos). Esto permite planificar comunicación y expectativas para reducir malentendidos y retrabajo.
- Pre-entrega: Brief de Proyecto y Mapa de Stakeholders: Entregable enfocado en dejar claro el Problem Statement, la Propuesta de valor, la matriz Poder vs. Interés y una estrategia inicial de gestión de expectativas + riesgos.

**¡Muchas  
gracias!**